AUTOMATED SECURITY ASSESSMENT & REPORTING PIPELINE

รายงานการแก้ไขระบบ ขั้นที่ 8

README 2 ภาษา · session ฝั่ง server · รหัสผ่าน Redis · ตัวจำกัด login ที่ไม่หายเมื่อ restart
แก้ปัญหาที่เจอเมื่อรันจริงใน Docker และบน Windows · เก็บหนี้เทคนิค C7 ◍
ต่อจาก "รายงานการแก้ไขระบบ ขั้นที่ 7"

> ชื่อระบบ: **VulnCompass**
> โปรเจกต์: `C:\Allprojects\Projects`
> GitHub: `github.com/jjjecop/security-pipe-ine` (private)
> ช่วงเวลา: 19 ตุลาคม 2026 · 12 commit · test 110 → **128** ตัว

1. สรุปสั้นๆ

ขั้นที่ 7 ทำให้ระบบรับมือกับความผิดพลาดได้และรันได้ด้วย Docker คำสั่งเดียว ขั้นที่ 8 มี 3 ส่วน: (1) **README ภาษาอังกฤษและไทย** สำหรับคนที่เข้ามาดู repo, (2) **ใช้ Redis ปิดช่องโหว่ด้านการยืนยันตัวตน 3 จุด** ที่เกิดจากการเก็บสถานะไว้ในหน่วยความจำหรือใน cookie และ (3) **แก้ปัญหาที่เจอเฉพาะตอนใช้งานจริง** ซึ่ง test ทุกข้อผ่านแต่ระบบจริงยังมีปัญหา

เรื่อง	ก่อน (จบขั้นที่ 7)	หลัง (จบขั้นที่ 8)
README	ภาษาไทย ล้าสมัย ไม่มีหน้าเว็บ, Docker และ CI	**อังกฤษ + ไทย** มีภาพหน้าจอ, แผนภาพสถาปัตยกรรม, Security design, Threat model
กด logout	ลบแค่ cookie ในเบราว์เซอร์ **สำเนา cookie ใช้ต่อได้อีก 8 ชั่วโมง**	ลบ session ใน Redis **สำเนาทุกชุดใช้ไม่ได้ทันที**
ตัวจำกัด login	อยู่ในหน่วยความจำ **restart API แล้วตัวนับหาย**	อยู่ใน Redis ไม่หายเมื่อ restart
Redis	**ไม่มีรหัสผ่าน** (พึ่งการเปิดเฉพาะ localhost อย่างเดียว)	ต้องใส่รหัสผ่าน และ compose ไม่ยอมเปิดถ้าไม่ได้ตั้ง
Redis หรือฐานข้อมูลล่ม	ฐานข้อมูลล่ม → 503 · Redis ยังไม่ได้ใช้ตอน login	ทั้งคู่ → **503** ไม่แสดง host/port
login บน Windows	**ช้าประมาณ 6 วินาที** (IPv6 fallback)	0.04 วินาที
สร้าง container api ใหม่	**หน้าเว็บได้ 502** จนกว่าจะ restart nginx	nginx หา IP ใหม่ได้เอง
รายงานเก่าใน Docker	**404** (path แบบ `reports\...`)	เปิดได้
คำเตือนตอนรัน test	3 (มี `utcnow`)	2 (มาจาก library ภายนอกทั้งคู่)
จำนวน test	110	128

ข้อมูลที่ระบบเก็บใน Redis ตอนนี้

```
Redis DB 0 (รันในเครื่อง)  /  DB 1 (รันใน Docker)        ต้องใส่ REDIS_PASSWORD
  scan_queue                  งานที่รอสแกน                      (ขั้นที่ 5)
  scan_processing             งานที่ worker กำลังทำ            (ขั้นที่ 7, reliable queue)
  login_failures:<ip>:<user>  จำนวนครั้งที่ login ผิด หมดอายุ 15 นาที  (ขั้นที่ 8)
  session:<sha256(sid)>       user_id ของ session      หมดอายุ 8 ชั่วโมง  (ขั้นที่ 8)
```

2. C8: README ภาษาอังกฤษและไทย

1 โครงสร้าง

ส่วน	ทำไมวางไว้ตรงนั้น
ประโยคแรก + ภาพ Dashboard + คำสั่งเดียว	คนที่เข้ามาดู repo ส่วนใหญ่อ่านแค่ 30 วินาทีแรก
ระบบทำอะไร (4 scanner, 6 ขั้นของ pipeline)	อธิบายว่าระบบนี้มีอะไรมากกว่าแค่เรียกใช้ Semgrep: fingerprint, ติดตามสถานะ fixed, EPSS, risk score
Architecture (Mermaid)	GitHub วาดเป็นแผนภาพให้เอง ไม่ต้องทำรูปแยก และแก้ง่ายเมื่อระบบเปลี่ยน
Security design + Threat model	จุดขายหลักของโปรเจกต์ security มีทั้งเหตุการณ์รหัสผ่านหลุด (ขั้นที่ 6) และ trade-off เรื่อง `docker.sock` (ขั้นที่ 7)
Quick start / พัฒนาในเครื่อง / CLI / CI	สำหรับคนที่อยากลองรันเอง

`README.md` เป็นภาษาอังกฤษตามธรรมเนียมของ repo หลายภาษา (GitHub แสดงไฟล์นี้บนหน้าแรก) ส่วน `README.th.md` เป็นภาษาไทย และมีลิงก์สลับภาษาไว้ด้านบนทั้งสองไฟล์

2 ภาพหน้าจอ

มี 3 รูปใน `docs/images/` : Dashboard และหน้าการสแกน (แคปจากหน้าเว็บจริง ย่อเหลือกว้าง 1600 px) และหน้ารายงาน PDF ซึ่งใช้**หน้า 2 ของ PDF จริง**แทนหน้า 1 เพราะเห็นคำแนะนำการแก้ไขภาษาไทย, EPSS และหมวด OWASP ส่วนหน้า 1 มีแค่ตารางสรุป ก่อนแคปตรวจแล้วว่าไม่มีข้อมูลส่วนตัวบนจอ

3 ไม่เขียนตัวเลขที่ยังไม่ได้ตรวจ

ร่างแรกเขียนว่าผลสแกนตัวเองใน CI คือ "Trivy 0, Semgrep 0" แต่ตรวจได้แค่ในเครื่อง เพราะ repo เป็น private พอไปดูใน GitHub Actions เห็นว่าทุกรอบ**ขึ้นสีเขียว** แต่สีเขียวยืนยันได้แค่ว่า**ไม่เจอช่องโหว่ระดับ critical** ไม่ได้ยืนยันว่า Semgrep เหลือ 0 จึงเปลี่ยนข้อความเป็นสิ่งที่ยืนยันได้จริง: "ผ่าน gate ทุกครั้ง, Trivy ไม่เจอ library ที่มีช่องโหว่, คำเตือน Semgrep 2 ข้อตรวจแล้ว"

README ที่ใช้สมัครงานถูกอ่านโดยคนที่อาจถามต่อ ตัวเลขที่พิสูจน์ไม่ได้มีแต่ทำให้เสียความน่าเชื่อถือ

ไฟล์: `README.md` (เขียนใหม่), `README.th.md` (ใหม่), `docs/images/*.png` (ใหม่), `.env.example` · commit `a59a335` , `6fe7f0d`

3. ใช้ Redis ปิดช่องโหว่ด้านการยืนยันตัวตน

ทั้ง 3 เรื่องในบทนี้มีต้นเหตุเดียวกัน: **สถานะสำคัญด้าน security ถูกเก็บไว้ในที่ที่ server ควบคุมไม่ได้** (ในหน่วยความจำของ process ซึ่งหายเมื่อ restart หรือใน cookie ซึ่งอยู่ที่เบราว์เซอร์) ทางแก้คือย้ายไปเก็บใน Redis ที่ทุก process ใช้ร่วมกันได้
แต่ก่อนจะย้ายอะไรที่สำคัญเข้าไปเก็บ ต้องทำให้ Redis ปลอดภัยก่อน

4 รหัสผ่านให้ Redis (ต้องทำก่อน)

ปัญหา: ใครที่ต่อ Redis ได้ก็สั่งได้ทุกคำสั่ง เช่น ยัดงานสแกนปลอมเข้าคิวโดยไม่ผ่านการ login และ allowlist ของ API ตอนนั้นยังปลอดภัยเพราะเปิดแค่ `127.0.0.1` แต่หลัก **defense in depth** คือไม่ฝากความปลอดภัยไว้ที่ด่านเดียว

การตัดสินใจ	เหตุผล
`REDIS_PASSWORD` แยกจาก `REDIS_URL`	ไม่ต้อง encode อักขระพิเศษใน URL และ URL ไม่มีความลับติดอยู่ถ้าถูกพิมพ์ลง log
`${REDIS_PASSWORD:?...}` ใน compose	fail closed: ลืมตั้งรหัสแล้ว Docker จะหยุดพร้อมบอกสาเหตุ แทนที่จะเปิด Redis แบบไม่มีรหัสโดยไม่มีใครรู้
`sh -c 'exec redis-server --requirepass "$$REDIS_PASSWORD"'`	ให้ shell ใน container อ่านค่าเอง รหัสจึงไม่อยู่ในคำสั่งของ compose และ `ps` แสดงแค่ `redis-server *:6379`
`password=... or None`	ถ้าส่งรหัสว่างไป Redis จะตอบ error ทั้งที่ Redis ไม่ได้ตั้งรหัส (ทดสอบแล้ว)

สร้างรหัส: สุ่ม 48 ตัวอักษร hex แล้วต่อท้าย `.env` โดยไม่แสดงค่าออกมา ตรวจได้แค่ความยาวและว่ามี 1 บรรทัด สำรอง `.env` ไว้ก่อนแก้ และลบไฟล์สำรองทิ้งทันทีหลังสำเร็จ เพื่อไม่ให้ความลับค้างอยู่ที่อื่น

ผล: `redis-cli PING` โดยไม่ใส่รหัส → `NOAUTH Authentication required.`

ไฟล์: `docker-compose.yml`, `scan_queue.py`, `.env.example`, `tests/test_scan_queue.py` · commit `25f0028`

5 ตัวจำกัด login เก็บใน Redis

ปัญหา: ตัวนับครั้งที่ login ผิดอยู่ในหน่วยความจำของ API พอ restart ก็หายหมด ผู้โจมตีที่โดนล็อกเดารหัสต่อได้ทันที และถ้ามี API หลายตัว แต่ละตัวจะนับแยกกัน

```
record_failure:  INCR  login_failures:<ip>:<user>
                 EXPIRE login_failures:<ip>:<user> 900 NX
is_blocked:      GET ... >= 5
reset:           DEL ...         (login สำเร็จ)
```

การตัดสินใจ	เหตุผล
`EXPIRE ... NX`	ตั้งเวลาหมดอายุครั้งเดียวตอนผิดครั้งแรก ถ้าต่อเวลาใหม่ทุกครั้งที่ผิด คนที่ค่อยๆ เดาช้าๆ จะทำให้เจ้าของบัญชีจริงโดนล็อกได้เรื่อยๆ
`INCR` ก่อนแล้วค่อย `EXPIRE`	ร่างแรกใช้ `SET NX EX` แล้วค่อย `INCR` แต่ถ้า key หมดอายุพอดีระหว่าง 2 คำสั่ง `INCR` จะสร้าง key ใหม่ที่**ไม่มีวันหมดอายุ** แบบใหม่ถ้าตั้งเวลาไม่สำเร็จ ครั้งถัดไปจะตั้งให้เอง
Redis ใช้ไม่ได้ → 503	fail closed: ถ้านับไม่ได้แล้วยังยอมให้ login ผู้โจมตีแค่ทำให้ Redis ล่มก็เดารหัสได้ไม่จำกัด
ใช้ client ตัวเดียวซ้ำ	client ของ redis-py มี connection pool ในตัว ไม่ต้องต่อใหม่ทุกครั้ง

ผลจริงผ่าน `:8080`: ผิดครั้งที่ 1–5 → 401 · ครั้งที่ 6 → 429 · **restart api แล้วยัง 429** (ก่อนแก้จะกลับเป็น 401)

ไฟล์: `auth.py`, `main.py`, `tests/fake_redis.py`, `tests/test_auth.py`, `tests/test_api_login.py`, `tests/test_db_unavailable.py` · commit `c5f1ca9`

6 Session เก็บฝั่ง server

ปัญหา: cookie เดิมเก็บ `{"user_id": 1}` ไว้ข้างใน (มีลายเซ็นกันการแก้ไข) server จึงไม่ต้องจำอะไรเลย แต่**ยกเลิก cookie ไม่ได้**: กด logout แล้วเบราว์เซอร์แค่ลบ cookie ของตัวเอง ถ้า cookie เคยถูกขโมยไป สำเนานั้นยังใช้ได้อีกถึง 8 ชั่วโมง

	เดิม (stateless)	ใหม่ (server-side)
ใน cookie	`{"user_id": 1}`	`{"sid": "<รหัสสุ่ม 43 ตัว>"}`
ใน Redis	-	`session:<sha256(sid)>` → `1` หมดอายุ 8 ชั่วโมง
logout	ลบ cookie ในเบราว์เซอร์	ลบใน Redis ด้วย สำเนาทุกชุดใช้ไม่ได้ทันที
cookie เก่าที่มี user_id	-	ใช้ไม่ได้ ทุกคน login ใหม่ 1 ครั้ง (ตั้งใจ)

ทำไมเก็บ hash ของ sid: ถ้าข้อมูลใน Redis หลุด ก็เอาไปทำ cookie ที่ใช้งานได้ไม่ได้ เป็นหลักเดียวกับที่ไม่เก็บรหัสผ่านตรงๆ

เพิ่ม handler ให้ `RedisError` ทั้งแอป ตอบ 503 พร้อมข้อความทั่วไปและบันทึกรายละเอียดใน log แบบเดียวกับฐานข้อมูลใน C7a

test 7 ข้อ รวม test ที่สร้าง cookie แบบเดียวกับที่ `SessionMiddleware` เซ็นให้เบราว์เซอร์ (`itsdangerous.TimestampSigner`) เพื่อทดสอบผ่าน HTTP จริง: login ได้ → logout → **สำเนา cookie ได้ 401** และตอน Redis ล่มได้ 503 โดยไม่มี `127.0.0.1` ในคำตอบ

ผลจริงผ่าน `:8080`: `/auth/me` ได้ admin → logout → สำเนา cookie ได้ **401** · session ที่เหลือใน Redis = 0 · ปิด Redis → **503** ข้อความทั่วไป · เปิด Redis กลับมา → worker ต่อได้เองโดยไม่ต้อง restart

ไฟล์: `auth.py`, `main.py`, `tests/fake_redis.py`, `tests/test_sessions.py` (ใหม่) · commit `fb98a30`

> **Trade-off ที่ยอมรับ:** session แบบเก็บฝั่ง server ทำให้ทุก request ต้องถาม Redis (ใช้เวลาไม่ถึง 1 ms ในเครื่องเดียวกัน) และ Redis กลายเป็นส่วนที่ขาดไม่ได้ของการ login แลกกับการที่ logout ยกเลิก session ได้จริง ซึ่งสำคัญกว่าสำหรับเครื่องมือ security

4. ปัญหาที่เจอเฉพาะตอนรันจริง

ทุกปัญหาในบทนี้ test ผ่านหมด แต่ระบบจริงยังมีปัญหาอยู่ ทั้งหมดเจอเพราะลองใช้งานจริงหลังแก้ทุกครั้ง

7 รายงานเก่าเปิดใน Docker ไม่ได้

อาการ: รายงานที่สร้างตอนรันบน Windows ถูกบันทึกเป็น `reports\NodeGoat_...\security_report.pdf` Linux ไม่ถือว่า `\` เป็นตัวคั่นโฟลเดอร์ ทั้งก้อนจึงกลายเป็นชื่อไฟล์เดียว → 404

แก้: แปลง `\` เป็น `/` ก่อนเปิดไฟล์ ซึ่งใช้ได้ทั้ง 2 ระบบ

ยังปลอดภัยไหม: หลังแปลง `reports\..\.env` จะกลายเป็น path ที่ชี้ออกไปนอกโฟลเดอร์จริงๆ จึงต้องพึ่งการตรวจ `is_path_inside` ในบรรทัดถัดไป และมี test ยืนยันว่าหลุดไม่ได้

test บน Linux: บน Windows test ผ่านตั้งแต่ก่อนแก้ จึงรันใน container: ก่อนแก้ `1 failed` หลังแก้ `5 passed` ส่วน CI รันบน Ubuntu จึงจับได้ถ้ามีคนลบบรรทัดนี้ในอนาคต

commit `f560ec3`

8 รายงานไปอยู่ผิดที่เมื่อรันจากโฟลเดอร์อื่น

อาการ: `output_dir="reports"` เป็น path สัมพัทธ์ ขึ้นกับโฟลเดอร์ที่เปิดโปรแกรม ถ้ารันจากที่อื่น รายงานจะไปอยู่ที่นั่นแล้ว API หาไม่เจอ

แก้: `REPORTS_DIR` ผูกกับโฟลเดอร์ของโปรเจกต์ และเพิ่ม `path_for_database()` บันทึกลงฐานข้อมูลแบบนับจากโฟลเดอร์โปรเจกต์และใช้ `/` ถ้าเก็บเป็น path เต็ม (`C:\Allprojects\...`) จะกลับไปเปิดใน Docker ไม่ได้เหมือนข้อ 7

ผล: งาน #50 ใน Docker บันทึกเป็น `reports/NodeGoat_20261019_163027/...` เปิดได้ทั้งจาก API ใน Docker และ Python บน Windows

commit `06d1f63`

9 nginx ส่งไปที่ IP เก่าของ api (502)

อาการ: หลัง build api ใหม่ หน้าเว็บ `:8080` ได้ 502 log เขียนว่า `connect() failed ... upstream: "http://172.18.0.5:8000"`

สาเหตุ: nginx หา IP ของชื่อ `api` ครั้งเดียวตอนเปิด แต่ container ใหม่ได้ IP ใหม่

แก้: `resolver 127.0.0.11 valid=10s` (DNS ภายในของ Docker) และเก็บที่อยู่ไว้ในตัวแปร nginx จะหา IP ใหม่ทุก 10 วินาที ต้องใช้ `rewrite` ตัด `/api` แทน เพราะ `proxy_pass` ที่ใช้ตัวแปรจะไม่แทนที่ URI ให้

ทดสอบ: บังคับสร้าง api ใหม่จนได้ IP ใหม่ (`172.18.0.4`) → ยังเรียกได้ปกติ

บทเรียน: ปัญหานี้น่าจะเกิดตั้งแต่ C7e แล้ว แต่เพิ่งเห็นตอนเช็กผลหลัง rebuild ทุกครั้งที่แก้ระบบ ควรลองเปิดผ่านหน้าเว็บจริงด้วย

commit `d92a10b`

10 ต่อ Redis และ Postgres ช้า 2 วินาทีทุกครั้ง

เจอได้อย่างไร: ตอนทดสอบตัวจำกัด login กับ Redis จริง TTL ลดจาก 900 เหลือ 890 ทั้งที่เรียกแค่ 5 ครั้ง จึงวัดเวลา:

ต่อด้วย	เวลา
Redis ผ่าน `localhost` (client ใหม่)	**2.04 วินาที**
Redis ผ่าน `127.0.0.1`	**0.007 วินาที**
Postgres ผ่าน `localhost`	**2.07 วินาที**
Postgres ผ่าน `127.0.0.1`	**0.009 วินาที**

สาเหตุ: บน Windows `localhost` จะลอง IPv6 (`::1`) ก่อน แต่ตั้งแต่ขั้นที่ 6 เราให้ Docker เปิด port แค่ `127.0.0.1` (IPv4) จึงต้องรอให้ IPv6 ล้มเหลวก่อน **เป็นผลข้างเคียงของการแก้ด้าน security ในขั้นก่อน**
แก้: ค่าเริ่มต้นใน `db.py` , `scan_queue.py` , `.env.example` เป็น `127.0.0.1` ไม่ต้องแก้ `.env` เพราะไม่ได้ตั้งค่าเหล่านี้ไว้ · login จาก ~6 วินาที เหลือ 0.04 วินาที

commit `3916e00`

11 งานเล็กที่เก็บไปด้วย

งาน	รายละเอียด
`datetime.utcnow()` → `utc_now()` (`33a3f75`)	Python จะลบ `utcnow` ในอนาคต ฟังก์ชันใหม่คืนค่า**แบบเดียวกับเดิมทุกอย่าง** (UTC ไม่มี timezone) เพราะคอลัมน์ในฐานข้อมูลไม่มี timezone และหน้าเว็บเติม `Z` เองตอนแสดงผล ถ้าเปลี่ยนรูปแบบ ข้อมูลเก่ากับใหม่จะปนกันและเวลาอาจคลาดไป 7 ชั่วโมง
`is_repo_allowed` → `is_path_inside` (`9affea2`)	ใช้ตรวจทั้งโฟลเดอร์ที่สแกนและไฟล์รายงาน ชื่อเดิมทำให้ `is_repo_allowed(path, [REPORTS_DIR])` อ่านแล้วงง
favicon (`19a6864`)	เข็มทิศสีฟ้าเดียวกับปุ่มในเว็บ แทนโลโก้ Vite สีม่วง ตรวจที่ขนาด 16 px และบนแถบแท็บสีเข้มแล้ว

5. ปัญหาด้านขั้นตอนการทำงาน และบทเรียน

5.1 แท็บค้างใน VS Code เขียนทับไฟล์ ร้ายแรงที่สุดในรอบนี้

อาการ: ระหว่างรอ commit `main.py` ถูก save ทับด้วยเวอร์ชันที่**ไม่มี import 8 บรรทัดแรก** (`import os`, `from fastapi import ...`) และโค้ดที่เพิ่งแก้หายไปด้วย ถ้า commit ไปแบบนั้น API จะเปิดไม่ขึ้น
สาเหตุ: แท็บ `main.py` ที่ค้างมาตั้งแต่ C7a (ตอนนั้นมีตัว `ง` หลุดเข้าไปและ import หาย) ยังไม่ได้ปิด แล้วถูกกด Ctrl+S ทับไฟล์ที่แก้ใหม่
แก้: `git restore main.py` แล้วใส่การแก้กลับเข้าไป · ตรวจด้วย `import main` + `pytest` ก่อน commit
บทเรียน: ถ้ามีคนอื่น (หรือเครื่องมือ) แก้ไฟล์ให้ ต้องปิดแท็บที่เปิดค้างก่อน หรือเปิด Auto Save · ก่อน commit ให้ดู `git diff --stat` ทุกครั้ง ถ้าจำนวนบรรทัดไม่ตรงกับที่คาดไว้ให้หยุดก่อน

5.2 ปัญหาเล็กอื่นๆ

อาการ	สาเหตุและบทเรียน
git เตือน `LF will be replaced by CRLF` และ VS Code ดูเหมือนทั้งไฟล์ถูกแก้	คัดลอกไฟล์จากโฟลเดอร์ทดลองมาพร้อมการขึ้นบรรทัดแบบ Linux แก้โดยคงรูปแบบเดิมของแต่ละไฟล์ไว้ทุกครั้งที่เขียนทับ
api ใน Docker ต่อ Redis ไม่ได้หลังตั้งรหัส (`AuthenticationError`) ทั้งที่ Python ในเครื่องต่อได้	image ยังมี `scan_queue.py` ตัวเก่า โค้ดถูก copy เข้า image ตอน build จึงต้อง `--build` ใหม่ทุกครั้งที่แก้ Python
บอกว่า push แล้ว แต่ commit ล่าสุดบน GitHub ยังเป็นของรอบก่อน	ไม่ได้รัน `git add` / `git commit` push จึงไม่มีอะไรให้ส่ง ตรวจด้วย `git log` และ `git status` ก่อนบอกว่าเสร็จ
`docker compose exec ... --repo /targets/NodeGoat` กลายเป็น `C:/Program Files/Git/targets/...`	Git Bash แปลง path ที่ขึ้นต้นด้วย `/` ให้เอง ใช้ `MSYS_NO_PATHCONV=1` (PowerShell ไม่มีปัญหานี้)
test ข้อ login ตอนฐานข้อมูลล่ม ผ่าน**ด้วยเหตุผลที่ผิด**	หลังย้ายตัวนับไป Redis test นี้จะได้ 503 เพราะ Redis ต่อไม่ได้ (ใน CI ไม่มี Redis) ไม่ใช่เพราะฐานข้อมูลล่ม จึงให้ test ใช้ Redis ปลอม และรัน test ทั้งชุดแบบ**ไม่มี Redis**ด้วยทุกครั้ง

6. สถานะงาน และสิ่งที่ยังเหลือ

commit ในรอบนี้ (ทั้งหมดอยู่บน `main` และ push แล้ว)

```
a59a335  C8   Rewrite README in English and Thai with screenshots, architecture, and security design
f560ec3  C7◌ Open reports saved with Windows paths when running on Linux
33a3f75  C7◌ Replace deprecated datetime.utcnow with a utc_now helper
d92a10b  C7◌ Re-resolve the api container address in nginx so it survives api restarts
25f0028  C7◌ Require a password for Redis
3916e00  C7◌ Default to 127.0.0.1 to avoid a 2 second IPv6 fallback on Windows
c5f1ca9  C7◌ Keep login failure counts in Redis so restarts do not reset the limit
19a6864  C7◌ Replace the Vite favicon with a compass icon
9affea2  C7◌ Rename is_repo_allowed to is_path_inside since it also guards report files
06d1f63  C7◌ Write reports under the project folder regardless of the working directory
fb98a30  C7◌ Store sessions in Redis so logout revokes copied cookies
6fe7f0d  C8   Update README security design for Redis sessions, login limiter, and Redis password
```

สถานะ

ระดับ	งาน	สถานะ
▨	C7a–C7d (503, กู้งานค้าง, triage Semgrep, Docker)	**เสร็จ (ขั้นที่ 7)**
◌	path รายงาน, utcnow, รหัสผ่าน Redis, ตัวจำกัด login, session ฝั่ง server, reports แบบ absolute, เปลี่ยนชื่อฟังก์ชัน, favicon	**เสร็จ**
◌	คำเตือน `httpx` ใน TestClient	รอ library รองรับ
C8	README อังกฤษ + ไทย, ภาพหน้าจอ, สถาปัตยกรรม, threat model	**เสร็จ**
C8	วิดีโอ demo 1–2 นาที	ขั้นต่อไป

ไอเดียต่อยอด (ยังไม่ทำ)

เรื่อง	ทำไม / ทำอย่างไร
DAST ใน Docker แบบปลอดภัย	แยก ZAP เป็น service ที่เรียกผ่าน API แทนการเปิด `docker.sock`
worker หลายตัว	แยกรายการ processing ตามชื่อ worker (ตอนนี้ออกแบบไว้สำหรับตัวเดียว)
HTTPS	ถ้าจะเปิดให้คนอื่นใช้ผ่านเครือข่าย ต้องใส่ TLS ที่ nginx และตั้ง `SESSION_HTTPS_ONLY=true`
logout ทุกเครื่อง	เก็บรายการ session ต่อ user ใน Redis ทำให้ยกเลิกทุก session ได้ตอนเปลี่ยนรหัสผ่านหรือปิดบัญชี

7. สรุปให้อ่าน: รอบนี้ทำอะไร ทำไม และได้อะไร

ทำอะไร

รอบนี้ทำให้โปรเจกต์**พร้อมให้คนอื่นเห็น**ด้วย README 2 ภาษาที่อธิบายทั้งระบบและการออกแบบด้าน security และทำให้**การยืนยันตัวตนแข็งแรงขึ้น** โดยย้ายสถานะสำคัญ 2 อย่าง (session และตัวนับ login ผิด) ไปไว้ใน Redis ที่ตั้งรหัสผ่านแล้ว นอกจากนั้นยังแก้ปัญหา 4 ข้อที่เจอเฉพาะตอนใช้งานจริงบน Windows และใน Docker

ทำไมต้องทำ

README: คนที่ดู portfolio อ่าน README ก่อนโค้ด และใช้เวลาไม่กี่นาที ถ้าไม่มี README ที่ดี งานที่ทำมาทั้งหมดจะไม่มีใครเห็น
Session และ rate limit: logout ที่ยกเลิก session ไม่ได้ และ rate limit ที่หายเมื่อ restart เป็นจุดอ่อนที่คนสาย security จะถามทันที เพราะเป็นโปรเจกต์ด้าน security
ปัญหาตอนรันจริง: ระบบที่ test ผ่านแต่หน้าเว็บได้ 502 หรือ login ช้า 6 วินาที คือระบบที่ใช้งานไม่ได้ในสายตาผู้ใช้

ได้อะไร

ด้าน	ผลที่ได้
ความปลอดภัย	logout ยกเลิก session ได้จริง · rate limit ไม่หายเมื่อ restart · Redis มีรหัสผ่าน · Redis ล่มแล้ว fail closed (503) · Redis เก็บแค่ hash ของ session id
ใช้งานได้จริง	รายงานเก่าเปิดใน Docker ได้ · nginx ไม่ 502 หลัง rebuild · login เร็วขึ้นจาก 6 วินาทีเหลือ 0.04 วินาที · รายงานไม่ไปอยู่ผิดที่
คุณภาพ	test 110 → 128 · test ทุกชุดรันได้ทั้งตอนมีและไม่มี Redis · test ที่ต้องจับบั๊กบน Linux ตรวจใน container แล้ว · คำเตือนของเราเองเหลือ 0
เอกสาร	README อังกฤษ + ไทย · ภาพหน้าจอ · แผนภาพ Mermaid · Security design · Threat model · ตัวเลขทุกตัวตรวจได้จริง

เรื่องที่เล่าตอนสัมภาษณ์งานได้

หัวข้อ	เล่าว่า
Stateless vs server-side session	signed cookie ไม่ต้องเก็บอะไรฝั่ง server แต่ยกเลิกไม่ได้ → ย้ายไป Redis แบบเก็บ hash ของ session id → พิสูจน์ด้วย test ที่ใช้สำเนา cookie หลัง logout แล้วได้ 401 → ยอมรับว่า Redis กลายเป็นส่วนที่ขาดไม่ได้
Rate limit ที่ถูกต้อง	ตัวนับในหน่วยความจำหายเมื่อ restart → ย้ายไป Redis → `EXPIRE NX` เพื่อไม่ให้ต่อเวลาทุกครั้ง → เปลี่ยนลำดับคำสั่งหลังเจอ race ที่จะทำให้ key ไม่มีวันหมดอายุ → fail closed
Defense in depth	Redis เปิดแค่ localhost อยู่แล้ว แต่ยังใส่รหัสผ่าน เพราะความปลอดภัยไม่ควรพึ่งด่านเดียว และทำให้ fail closed ถ้าลืมตั้งรหัส
ผลข้างเคียงของการแก้ security	เปิด port แค่ IPv4 (ขั้นที่ 6) ทำให้ `localhost` บน Windows ช้า 2 วินาที → เจอจากการสังเกต TTL ที่ลดลงผิดปกติ → วัดผลก่อนแก้ → แก้ที่ค่าเริ่มต้น
Cross-platform	path แบบ Windows ใน Linux, path สัมพัทธ์ที่ขึ้นกับ working directory, DNS ของ container ที่เปลี่ยนเมื่อ rebuild ทุกข้อ test ผ่านแต่ระบบจริงพัง
ความซื่อตรงของเอกสาร	เปลี่ยนข้อความใน README จาก "Semgrep 0" เป็นสิ่งที่พิสูจน์ได้จริง หลังพบว่า CI สีเขียวไม่ได้ยืนยันตัวเลขนั้น

สรุปในประโยคเดียว: จบขั้นที่ 8 VulnCompass มี README ที่อธิบายทั้งระบบและการออกแบบด้าน security ใน 2 ภาษา การยืนยันตัวตนยกเลิก session ได้จริงและจำกัดการเดารหัสได้แม้ restart และทำงานได้ราบรื่นทั้งบน Windows และใน Docker งานที่เหลือของ C8 คือวิดีโอ demo